ECSE 324: Computer Organization
Lab 1: Basic Assembly Language Programming

Abstract

This lab presents three different programming problems: (a) recursive function calls, (b) working with loops and arrays, and (c) sorting. Note that the exercises are in no particular order. You will demonstrate your code in a short video where you step through it, showing results in registers and memories; you will also submit a short report documenting your approach.

Summary of Deliverables

- Source code for each algorithm
- Demo video, no longer than five (5) minutes
- Report, no longer than four (4) pages (10 pt font, 1” margins)

Please submit the above in a single .zip archive, using the following file name conventions:

- Code: part_1-fact.s, part_1-fib.s, part_2.s, part_3.s
- Video: StudentID_FullName_Lab1_demo.{mov, mp4, etc}
- Report: StudentID_FullName_Lab1_report.pdf

Grading

Coming soon ...

Changelog

- 22-Jan-2021 Initial, incomplete revision.
- 25-Jan-2021 2D convolution in Part 2 revised.
- 27-Jan-2021 Part 2 updated: typos in example code fixed (kw, kh); you may assume that image and kernel are square. Output results were also revised.
- 29-Jan-2021 Part 2 updated: Fixed bugs in algorithm 2

© Brett H. Meyer

Part 1: Factorial, Fibonacci, and Function Calls

Note: until we discuss subroutines and the stack, it may be easier to start with Part 2.

In this part, your task is to implement two recursive procedures, one to calculate the factorial function, another to calculate Fibonacci numbers.

The subroutine calling convention

It is important to carefully respect subroutine calling conventions in order to prevent call stack corruption. The convention which we will use for calling a subroutine in ARM assembly is as follows.

The caller (the code that is calling a function) must:

- Move arguments into R0 through R3. (If more than four arguments are required, the caller should PUSH the arguments onto the stack.)
- Call the subroutine using BL Func.

The callee (the code in the function that is called) must:

- Move the return value into R0.
- Ensure that the state of the processor is restored to what it was before the subroutine call by POPing arguments off of the stack.
- Use BX LR to return to the calling code.

Note that the state of the processor can be saved and restored by pushing R4 through LR onto the stack at the beginning of the subroutine and popping R4 through LR off the stack at the end of the subroutine.

Factorial calculation using recursive subroutine calls

A recursive subroutine is a subroutine which calls itself. The C below describes a recursive function that calculates the factorial function for a given positive integer n:

```
int fact(int n) {
  if (n < 2)
    return 1;
  else
    return n * fact(n-1);
}
```

For example, fact(4) is computed as follows:

4 * fact(3) = 4 * 3 * fact(2) = 4 * 3 * 2 * fact(1) = 4 * 3 * 2 * 1 = 24.

Fibonacci calculation using recursive subroutine calls

You can calculate the nth Fibonacci number, Fib(n) (where Fib(0) = 1, Fib(1) = 1, Fib(2) = 2, Fib(3) = 3, Fib(4) = 5, ...), using recursion as follows:

```
int fib(int n) {
  if (n < 2)
    return 1;
  else
    return fib(n-1) + fib(n-2);
}
```

For example, Fib(4) is computed as follows:

Fib(4) = Fib(3) + Fib(2) = (Fib(2) + Fib(1)) + (Fib(1) + Fib(0))
= ((Fib(1) + Fib(0)) + 1) + (1 + 1) = (1 + 1 + 1) + (1 + 1) = 5.

Exercises

1. Write an assembly program that uses recursive function calls to compute the factorial of any positive integer.
2. Write an assembly program that uses recursive function calls to compute the nth Fibonacci number.

Part 2: 2D Convolution

In this problem you will be implementing the 2D convolution algorithm that is usually used in image processing applications. For example, 2D convolution is used to implement image filters in your favorite social media application and to detect objects using machine learning.

A straightforward implementation of 2D convolution in C consists of deeply nested for loops:

```
// Initialization
int fx [10][10] ={
    {183, 207, 128, 30, 109, 0, 14, 52, 15, 210},
    {228, 76, 48, 82, 179, 194, 22, 168, 58, 116},
    {228, 217, 180, 181, 243, 65, 24, 127, 216, 118},
    {64, 210, 138, 104, 80, 137, 212, 196, 150, 139},
    {155, 154, 36, 254, 218, 65, 3, 11, 91, 95},
    {219, 10, 45, 193, 204, 196, 25, 177, 188, 170},
    {189, 241, 102, 237, 251, 223, 10, 24, 171, 71},
    {0, 4, 81, 158, 59, 232, 155, 217, 181, 19},
    {25, 12, 80, 244, 227, 101, 250, 103, 68, 46},
    {136, 152, 144, 2, 97, 250, 47, 58, 214, 51}
    }; // 2D Image

int kx [5][5] = {
    {1,   1,  0,  -1,  -1},
    {0,   1,  0,  -1,   0},
    {0,   0,  1,   0,   0},
    {0,  -1,  0,   1,   0},
    {-1, -1,  0,   1,   1}
    }; // Kernel

int gx [10][10]; // Output Result

int iw = 10; // Image Width   = 10
int ih = 10; // Image Height  = 10
int kw =  5; // Kernel Width  = 5
int kh =  5; // Kernel Height = 5
int ksw = 2; // Kernel Width Stride  = (Kernel Width-1)/2
int khw = 2; // Kernel Height Stride = (Kernel Height-1)/2
```

```
// 2D Convolution Algorithm
for (int y = 0; y < ih ; y++) {
  for (int x = 0; x < iw ; x++) {
    int sum = 0;
    for (int i = 0; i < kw ; i++) {
      for (int j = 0; j < kh ; j++) {
        int temp1 = x+j -ksw;
        int temp2 = y+i -khw;
        if (temp1>=0 && temp1<=9 && temp2>=0 && temp2<=9)
          sum = sum + kx[j][i] * fx [temp1][temp2];
      }
    }

    gx[x][y] = sum;
  }
}

/* Output: Array = {
           {656 ,160 ,113 ,72 ,-51 ,-430 ,23 ,333 ,-70 ,-191 },
           {586 ,261 ,99 ,33 ,200 ,294 ,161 ,299 ,-378 ,-431},
           {217 ,631 ,482 ,311 ,86 ,-31 ,-30 ,-64 ,148 ,-9},
           {-68 ,256 ,485 ,319 ,-96 ,17 ,208 ,271 ,285 ,125},
           {-99 ,-77 ,404 ,691 ,354 ,-427 ,-389 ,0 ,211 ,205},
           {43 ,103 ,244 ,497 ,420 ,101 ,-142 ,123 ,67 ,38},
           {85 ,660 ,344 ,199 ,571 ,838 ,38 ,-468 ,-408 ,9 },
           {12 ,137 ,-40 ,-138 ,98 ,697 ,316 ,-295 ,55 ,215},
           {-170 ,-211 ,-282 ,88 ,507 ,409 ,352 ,235 ,222 ,208},
           {39 ,-142 ,-301 ,-351 ,92 ,72 ,-62 ,427 ,624 ,517},
   }
*/
```

You may assume that the input image and kernel are square.

Exercise

1. Write an assembly program that implements 2D convolution. Note that fx, image and kernel sizes, are all parameters passed to the program.

Part 3: Bubble Sort

In this part, your task is to implement the bubble sort algorithm. Below is an example of the bubble sort algorithm implemented in C and using pointers.

```
// Initialization
int size = 5;
int array[] = {-1, 23, 0, 12, -7};
int *ptr = &array[0];

// Bubble sort algorithm
for (int step = 0; step < size - 1; step++) {
  for (int i = 0; i < size - step - 1; i++) {

    // Sorting in ascending order.
    // To sort in descending order, change ">" to "<".
    if (*(ptr + i) > *(ptr + i + 1)) {
      // Swap if the larger element is in a later position.
      int tmp = *(ptr + i);
      *(ptr + i) = *(ptr + i + 1);
      *(ptr + i + 1) = tmp;
    }
  }
}
// Output: Array = {-7,  -1,  0,  12,  23}
```

Exercise

1. Write an assembly program that implements the bubble sort algorithm to sort a given array in ascending order. Note that size and array are both input parameters.

Deliverables

In your video, step through the execution of each of your algorithms. Clearly state and indicate where inputs are in memory, briefly narrate code behavior, and indicate the output of your code and where they are stored.

Your video will be graded by assessing, for each algorithm, the correctness of the observed behavior, and the correctness of your description of that behavior.

In your report, for each algorithm, describe:

- Describe your approach (e.g., how you used subroutines, the stack, etc)
- Challenges you faced, if any, and your solutions
- Shortcomings, possible improvements, etc

Your report will be graded by assessing, for each algorithm, your report's clarity, organization, and technical content.

Grading

Coming soon ...

Submission

Please submit, on MyCourses, your source code, video, and report in a single .zip archive, using the following file name conventions:

- Code: part_1-fact.s, part_1-fib.s, part_2.s, part_3.s
- Video: StudentID_FullName_Lab1_demo.{mov, mp4, etc}
- Report: StudentID_FullName_Lab1_report.pdf

© Brett H. Meyer